

---

# ROS2 PlanSys2 and Task Planning for Multiple Robots:

Preliminary Research Toward DEL-based Task Planning  
with EPDDL for Multi-Agent Robotics in Uncertain  
Environments

---

## Preliminary Research Document

Preliminary research in the context of epistemic multi-agent planning,  
grounded in Dynamic Epistemic Logic and the EPDDL framework

**Research Area:** Autonomous Robotics & Epistemic Planning  
**Keywords:** ROS2, PlanSys2, PDDL, DEL, EPDDL, Multi-Robot Systems  
**Document Type:** Preliminary Research / State of the Art  
**Version:** 1.1

*Haniel Ulises Vásquez Morales*

This document is a preliminary academic survey compiled in preparation for research on DEL-based epistemic task planning using EPDDL for multi-agent robotic systems operating under partial observability. It is not a finished publication.

## Abstract

The increasing deployment of autonomous robotic systems in complex, uncertain, and partially observable environments demands planning formalisms that transcend the capabilities of classical automated planning. This document constitutes a preliminary research survey covering two major pillars of the target research agenda: (i) the state of the art in robot task planning using the Robot Operating System 2 (ROS2), with particular focus on the PlanSys2 framework and its PDDL-based planning infrastructure; and (ii) the theoretical and conceptual foundations of Dynamic Epistemic Logic (DEL) and the Epistemic Planning Domain Definition Language (EPDDL), as introduced by Burigana and Fabiano for IPC 2026. The ultimate motivation of this survey is to chart a research path toward a DEL-based multi-agent task planning system for heterogeneous robot fleets operating under epistemic uncertainty. We discuss the architecture of ROS2 multi-robot systems, the structure and limitations of existing PDDL-based planners in robotics, the semantics of epistemic states and actions in DEL, and the syntactic mechanisms of EPDDL. Special attention is given to recent applied work applying DEL to heterogeneous robotic fleets under communication restrictions [15–17], and to the foundational treatment of DEL-based planning problems [18]. We conclude with an analysis of the integration challenges and open research questions at the intersection of these fields.

**Keywords:** ROS2, PlanSys2, PDDL, task planning, multi-robot systems, Dynamic Epistemic Logic, epistemic planning, EPDDL, partial observability, multi-agent systems, active inference, theory of mind.

## Contents

---

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                   | <b>4</b>  |
| <b>2</b> | <b>ROS2: Architecture and Multi-Robot Infrastructure</b>              | <b>5</b>  |
| 2.1      | Overview and Design Principles . . . . .                              | 5         |
| 2.2      | DDS and the Communication Model . . . . .                             | 5         |
| 2.3      | Node Lifecycle and Managed Nodes . . . . .                            | 6         |
| 2.4      | Multi-Robot Namespacing and Coordination . . . . .                    | 6         |
| 2.5      | ROS2 Actions and Task Execution . . . . .                             | 6         |
| <b>3</b> | <b>PlanSys2: Task Planning in ROS2</b>                                | <b>7</b>  |
| 3.1      | Architecture Overview . . . . .                                       | 7         |
| 3.2      | Domain and Problem Management . . . . .                               | 7         |
| 3.3      | Plan Generation and PDDL Solvers . . . . .                            | 7         |
| 3.4      | Execution via Behavior Trees . . . . .                                | 8         |
| 3.5      | A Practical PlanSys2 Task: Package Transport in ROS2 Humble/Jazzy . . | 8         |
| 3.5.1    | Scenario Description . . . . .                                        | 8         |
| 3.5.2    | Package Structure . . . . .                                           | 9         |
| 3.5.3    | PDDL Domain . . . . .                                                 | 9         |
| 3.5.4    | PDDL Problem . . . . .                                                | 10        |
| 3.5.5    | C++ Action Executors . . . . .                                        | 11        |
| 3.5.6    | CMakeLists.txt . . . . .                                              | 14        |
| 3.5.7    | Launch File . . . . .                                                 | 15        |
| 3.5.8    | Building and Running . . . . .                                        | 16        |
| 3.5.9    | Expected Execution Behavior . . . . .                                 | 16        |
| 3.6      | Known Limitations of PlanSys2 . . . . .                               | 17        |
| <b>4</b> | <b>Multi-Robot Systems in ROS2</b>                                    | <b>19</b> |
| 4.1      | Architectural Patterns for Multi-Robot Systems . . . . .              | 19        |
| 4.2      | Multi-Robot Navigation and Coordination . . . . .                     | 19        |
| 4.3      | Task Allocation and Multi-Agent Planning . . . . .                    | 19        |
| 4.4      | Shared World Modeling . . . . .                                       | 19        |
| <b>5</b> | <b>Dynamic Epistemic Logic: Foundations for Epistemic Planning</b>    | <b>21</b> |
| 5.1      | Motivation: From Classical to Epistemic Planning . . . . .            | 21        |
| 5.2      | Epistemic States and Kripke Semantics . . . . .                       | 21        |
| 5.3      | Common Knowledge . . . . .                                            | 21        |
| 5.4      | Epistemic Actions and the Product Update . . . . .                    | 22        |
| 5.5      | Types of Epistemic Actions . . . . .                                  | 22        |

|          |                                                                                  |           |
|----------|----------------------------------------------------------------------------------|-----------|
| 5.6      | Epistemic Planning Tasks . . . . .                                               | 22        |
| 5.7      | DEL in Robotics: From Theory to Application . . . . .                            | 23        |
| 5.8      | Belief and Empathy States in Multi-Robot DEL . . . . .                           | 24        |
| 5.9      | Theory of Mind, Higher-Order Reasoning, and Active Epistemic Inference . . . . . | 25        |
| 5.10     | DEL Planning Problem Variants and Structural Properties . . . . .                | 27        |
| <b>6</b> | <b>EPDDL: The Epistemic Planning Domain Definition Language</b>                  | <b>28</b> |
| 6.1      | Motivation and Design Philosophy . . . . .                                       | 28        |
| 6.2      | Language Structure . . . . .                                                     | 28        |
| 6.3      | EPDDL Syntax: A Concrete Example . . . . .                                       | 28        |
| 6.3.1    | EPDDL Domain . . . . .                                                           | 28        |
| 6.3.2    | EPDDL Problem . . . . .                                                          | 29        |
| 6.4      | Modal Operators in EPDDL . . . . .                                               | 30        |
| 6.5      | Initial State Representation . . . . .                                           | 31        |
| 6.6      | DEL Fragments via Action Type Libraries . . . . .                                | 31        |
| 6.7      | Abstract Event Models and Action Types . . . . .                                 | 31        |
| <b>7</b> | <b>Integration: Toward a DEL-Based Planning System for ROS2</b>                  | <b>33</b> |
| 7.1      | Conceptual Architecture . . . . .                                                | 33        |
| 7.2      | State Space and Computational Challenges . . . . .                               | 34        |
| 7.3      | Perception-to-Epistemic-State Translation . . . . .                              | 34        |
| 7.4      | Communication Actions as Epistemic Events . . . . .                              | 35        |
| 7.5      | Open Research Questions . . . . .                                                | 35        |
| <b>8</b> | <b>Conclusion</b>                                                                | <b>36</b> |

# 1 Introduction

---

Modern autonomous robotic systems are increasingly expected to operate in environments that are shared with other agents whether human operators, other autonomous robots, or dynamically changing physical conditions where the state of the world is not fully and reliably observable. A warehouse robot fleet, a team of search-and-rescue drones, or a collaborative assembly line all require their constituent agents to reason not only about what is true in the world, but also about what each agent *knows*, what each agent *believes*, and what each agent knows or believes about the epistemic states of others. Classical task planning formalisms, such as PDDL (Planning Domain Definition Language), remain the de facto standard in practical robotic systems and are supported by major middleware frameworks such as ROS2 through tools like PlanSys2. However, classical planning fundamentally assumes a fully observable, deterministic environment and a single omniscient planning agent assumptions that are routinely violated in realistic multi-robot deployments.

The objective of this document is to establish a comprehensive preliminary understanding of two converging research areas: the engineering and middleware infrastructure for task planning in ROS2 multi-robot systems, and the theoretical framework provided by Dynamic Epistemic Logic (DEL) and its associated language, EPDDL, for epistemic multi-agent planning. Together, these two pillars form the foundation upon which a future system can be built: one capable of reasoning about the knowledge and beliefs of individual robots, performing multi-agent coordination under partial observability, and expressing rich epistemic goals such as “robot *A* knows whether corridor *c* is blocked,” or “it is common knowledge among all robots that the package has been delivered.”

Section 2 introduces ROS2 as a middleware architecture, covering its communication model, multi-robot support, and lifecycle management. Section 3 examines PlanSys2, the primary task planning framework for ROS2, including its PDDL integration, executor architecture, and a concrete worked example using ROS2 Humble/Jazzy C++. Section 4 surveys the state of the art in multi-robot coordination and planning within ROS2. Section 5 provides an accessible but technically grounded introduction to Dynamic Epistemic Logic, including its application to robotics [15–18]. Section 6 presents EPDDL, its syntax, and action type libraries. Section 7 analyzes the integration challenges and proposes a conceptual architecture for a DEL-based ROS2 planning system. Section 8 summarizes findings and outlines open research questions.

## 2 ROS2: Architecture and Multi-Robot Infrastructure

### 2.1 Overview and Design Principles

The Robot Operating System 2 (ROS2) is an open-source middleware framework designed to provide a flexible and modular infrastructure for robotic software development [1]. Unlike its predecessor ROS1, which relied on a centralized master node, ROS2 is built atop the Data Distribution Service (DDS) standard, providing a fully decentralized, quality-of-service-aware publish-subscribe communication layer. This architectural shift makes ROS2 fundamentally more suited to multi-robot deployments, real-time systems, and industrial applications.

The core communication abstractions in ROS2 are *nodes*, *topics*, *services*, and *actions*. Topics implement an asynchronous, many-to-many publish-subscribe pattern. Services implement synchronous request-reply communication. Actions extend services with long-running asynchronous goals, periodic feedback, and cancellation.

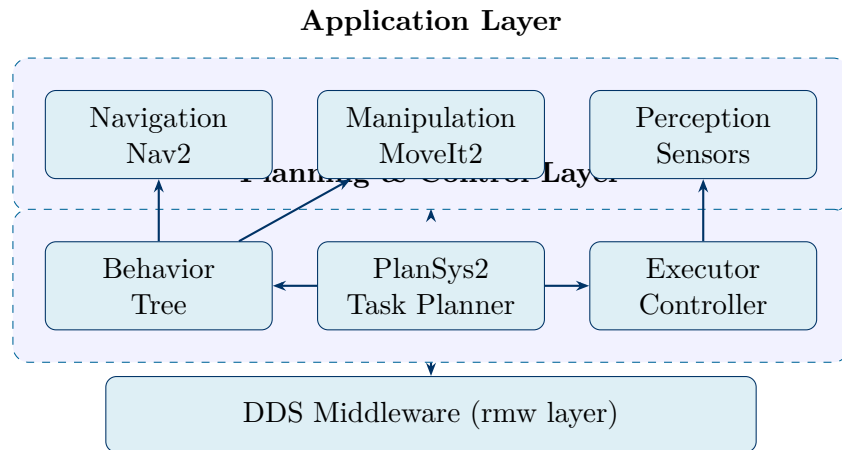


Figure 1: Simplified layered architecture of a ROS2 robot system. PlanSys2 operates within the Planning & Control Layer, interfacing with navigation, manipulation, and perception via behavior trees and action servers, all underpinned by DDS.

### 2.2 DDS and the Communication Model

DDS implements a data-centric publish-subscribe model where data flows between publishers and subscribers through a shared data space the *Global Data Space* without a centralized broker. Quality of Service (QoS) policies govern reliability, durability, deadline, and liveness. For multi-robot systems, DDS offers native distributed discovery: robots on the same network segment automatically discover each other without manual configuration, mediated through the *DDS Domain* abstraction that partitions communication into independent contexts.

### 2.3 Node Lifecycle and Managed Nodes

ROS2 introduced a formal node lifecycle management protocol with states (Unconfigured, Inactive, Active, Finalized) and defined transitions. From a task planning perspective, the lifecycle model provides a principled mechanism for activating and deactivating capabilities on demand, enabling a planning system to activate a navigation capability only when an action requiring it is dispatched.

### 2.4 Multi-Robot Namespacing and Coordination

ROS2 addresses multi-robot naming through a hierarchical namespace mechanism. Each robot is assigned a unique namespace prefix (e.g., `/robot1`, `/robot2`), and all its nodes, topics, services, and actions are scoped under that prefix. This allows multiple robot instances to coexist on the same DDS domain without topic collisions and enables a centralized planning node to address individual robots by namespace. Namespacing alone is, however, insufficient for true multi-robot coordination, which additionally requires shared state representation, inter-robot protocols, and synchronized action dispatch.

### 2.5 ROS2 Actions and Task Execution

The ROS2 action protocol is the primary mechanism for integrating task planning with robot execution. An action server accepts goal requests from action clients, provides periodic feedback, and returns a final result. This maps directly to the classical planning loop: the planner emits a sequence of actions, each dispatched as a goal to the corresponding action server, with execution monitored by the planner or a behavior executive. Action state management (ACCEPTED, EXECUTING, CANCELING, SUCCEEDED, ABORTED, CANCELED) is handled transparently by the ROS2 action infrastructure.



### 3 PlanSys2: Task Planning in ROS2

#### 3.1 Architecture Overview

PlanSys2 is the primary open-source framework for PDDL-based task planning in ROS2 [2]. It provides a modular, multi-node architecture integrating domain and problem management, plan generation via external PDDL solvers, and plan execution via a behavior-tree-based action dispatcher.

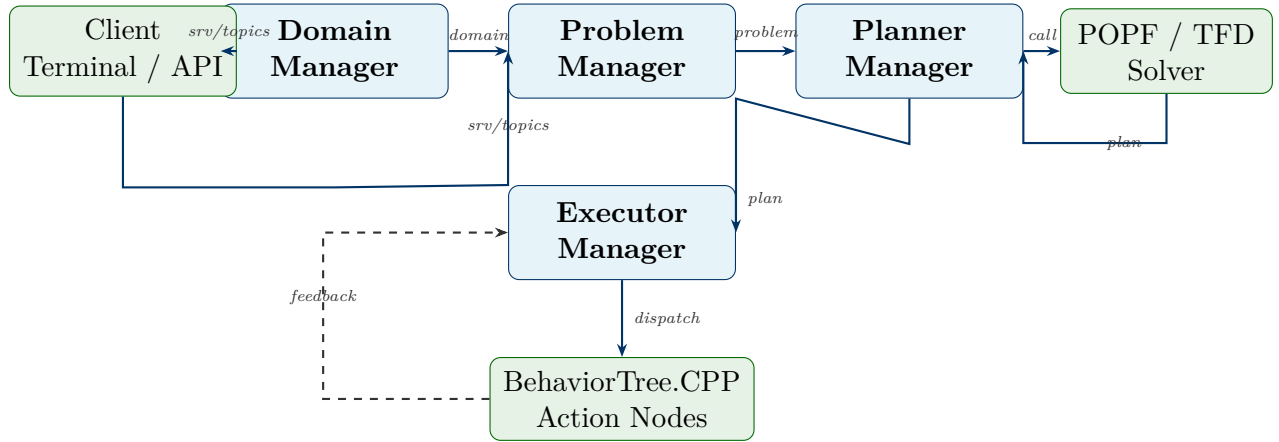


Figure 2: Internal architecture of PlanSys2. Domain Manager, Problem Manager, Planner Manager, and Executor Manager are separate ROS2 nodes. An external solver (POPF/TFD) is called by the Planner Manager, and execution is carried out through BehaviorTree.CPP action nodes.

PlanSys2 decouples domain management, problem management, planning, and execution into separate ROS2 nodes following the microservice principle. Each component can be independently replaced, updated, or scaled.

#### 3.2 Domain and Problem Management

The Domain Manager maintains the PDDL domain specification. The Problem Manager maintains the dynamic planning problem the current set of ground predicates (the world state), instantiated objects, and the current goal specification and is the primary interface through which the robot system updates its world model as perception events are received. Both managers expose ROS2 services and topics, allowing any node to add/remove predicates, objects, and update goals.

#### 3.3 Plan Generation and PDDL Solvers

PlanSys2 delegates planning to external PDDL planners via a plugin architecture. The most commonly used solvers are POPF [7] and TFD (Temporal Fast Downward), both supporting temporal PDDL with durative actions. The planning loop is triggered by a goal

update or world state change that renders the current plan invalid: the system serializes the domain and problem, calls the solver, receives the plan, and passes it to the Executor Manager.

### 3.4 Execution via Behavior Trees

Plan execution in PlanSys2 is mediated by the BehaviorTree.CPP library [8]. Each PDDL action is associated with a behavior tree node implementing execution logic, pre-condition verification, and post-condition assertion. The Executor Manager constructs a behavior tree from the plan and runs it, monitoring each action for success, failure, or timeout. The behavior tree is constructed statically from the plan; replanning must be explicitly triggered when execution deviates from expectations.

### 3.5 A Practical PlanSys2 Task: Package Transport in ROS2 Humble/Jazzy

To ground the preceding architectural description concretely, we walk through a complete PlanSys2 task package for ROS2 Humble (or Jazzy) targeting a logistics scenario: a single mobile robot must transport two packages between three named locations in a warehouse. We cover the scenario, the package structure, the PDDL domain and problem files, the C++ action executor implementations, the CMake build configuration, the launch file, and the expected execution behavior illustrated with diagrams.

#### 3.5.1 Scenario Description

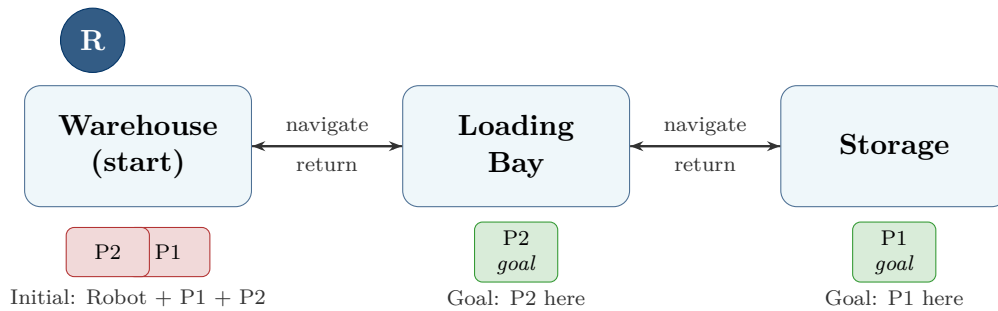


Figure 3: Warehouse scenario: the robot (R) starts at the Warehouse; P1 and P2 are both at the Warehouse. The goal is to deliver P1 to Storage and P2 to the Loading Bay.

The environment has three locations `warehouse`, `loading_bay`, and `storage` connected bidirectionally. The robot begins at the warehouse. Both packages begin at the warehouse. The plan requires picking up each package, navigating to its destination, dropping it off, and returning to the start.

### 3.5.2 Package Structure

The task is implemented as a standard ROS2 package (compatible with both Humble and Jazzy). The directory layout is as follows.

```

1 transport_task/
2 |-- CMakeLists.txt
3 |-- package.xml
4 |-- pddl/
5 |   |-- transport_domain.pddl
6 |   '-- transport_problem.pddl
7 |-- src/
8 |   |-- navigate_action_node.cpp
9 |   |-- pick_up_action_node.cpp
10 |   '-- drop_off_action_node.cpp
11 |-- launch/
12 |   '-- transport_task.launch.py
13 '-- config/
14   '-- plansys2_params.yaml

```

Listing 1: Package directory structure for the transport task.

### 3.5.3 PDDL Domain

```

1 (define (domain transport)
2   (:requirements :strips :typing :durative-actions :negative-
3     preconditions)
4
5   (:types
6     robot location package
7   )
8
9   (:predicates
10     (robot-at ?r - robot ?l - location)
11     (package-at ?p - package ?l - location)
12     (carrying ?r - robot ?p - package)
13     (free ?r - robot)
14     (connected ?from - location ?to - location)
15   )
16
17   (:durative-action navigate
18     :parameters (?r - robot ?from - location ?to - location)
19     :duration (= ?duration 10)
20     :condition (and
21       (at start (robot-at ?r ?from))
22       (at start (connected ?from ?to))
23       (over all (free ?r))
24     )
25   )
26 )

```

```

24   :effect (and
25     (at start (not (robot-at ?r ?from)))
26     (at end   (robot-at ?r ?to))
27   )
28 )
29
30 (:durative-action pick-up
31   :parameters (?r - robot ?p - package ?l - location)
32   :duration (= ?duration 5)
33   :condition (and
34     (at start (robot-at ?r ?l))
35     (at start (package-at ?p ?l))
36     (at start (free ?r))
37   )
38   :effect (and
39     (at start (not (package-at ?p ?l)))
40     (at start (not (free ?r)))
41     (at end   (carrying ?r ?p))
42   )
43 )
44
45 (:durative-action drop-off
46   :parameters (?r - robot ?p - package ?l - location)
47   :duration (= ?duration 5)
48   :condition (and
49     (at start (robot-at ?r ?l))
50     (at start (carrying ?r ?p))
51   )
52   :effect (and
53     (at start (not (carrying ?r ?p)))
54     (at end   (package-at ?p ?l))
55     (at end   (free ?r))
56   )
57 )
58 )

```

Listing 2: PDDL domain for the warehouse transport task (pddl/transport\_domain.pddl).

### 3.5.4 PDDL Problem

```

1 (define (problem transport-1)
2   (:domain transport)
3
4   (:objects
5     robot1                - robot
6     warehouse loading-bay storage - location

```

```

7      parcel1 parcel2                - package
8  )
9
10 (:init
11   (robot-at robot1 warehouse)
12   (package-at parcel1 warehouse)
13   (package-at parcel2 warehouse)
14   (free robot1)
15   (connected warehouse loading-bay)
16   (connected loading-bay warehouse)
17   (connected loading-bay storage)
18   (connected storage loading-bay)
19 )
20
21 (:goal (and
22   (package-at parcel1 storage)
23   (package-at parcel2 loading-bay)
24   (robot-at robot1 warehouse)
25 ))
26 )

```

Listing 3: PDDL problem instance (pddl/transport\_problem.pddl).

### 3.5.5 C++ Action Executors

Each durative action in the PDDL domain corresponds to a C++ class implementing `plansys2::ActionExecutorClient`. The following listings show the three executors. All three follow the same pattern: on activation they simulate the action with a progress timer, and on success they call `finish(true, 1.0, "succeeded")`. In a physical deployment the timer would be replaced by actual hardware calls (Nav2 goals, gripper commands, etc.).

```

1  #include <memory>
2  #include <string>
3  #include "plansys2_executor/ActionExecutorClient.hpp"
4  #include "rclcpp/rclcpp.hpp"
5  #include "rclcpp_lifecycle/lifecycle_node.hpp"
6
7  using namespace std::chrono_literals;
8
9  class NavigateAction : public plansys2::ActionExecutorClient
10 {
11 public:
12     NavigateAction() : plansys2::ActionExecutorClient("navigate", 250ms)
13     {
14         progress_ = 0.0f;

```

```

15     }
16
17 private:
18     void do_work() override
19     {
20         if (progress_ < 1.0f) {
21             progress_ += 0.05f;
22             send_feedback(progress_, "navigating...");
23         } else {
24             finish(true, 1.0f, "navigation completed");
25             progress_ = 0.0f;
26         }
27     }
28
29     float progress_;
30 };
31
32 int main(int argc, char ** argv)
33 {
34     rclcpp::init(argc, argv);
35     auto node = std::make_shared<NavigateAction>();
36     node->set_parameter(rclcpp::Parameter("action_name", "navigate"));
37     node->trigger_transition(
38         lifecycle_msgs::msg::Transition::TRANSITION_CONFIGURE);
39     node->trigger_transition(
40         lifecycle_msgs::msg::Transition::TRANSITION_ACTIVATE);
41     rclcpp::spin(node->get_node_base_interface());
42     rclcpp::shutdown();
43     return 0;
44 }

```

Listing 4: Navigate action executor (src/navigate\_action\_node.cpp).

```

1 #include <memory>
2 #include <string>
3 #include "plansys2_executor/ActionExecutorClient.hpp"
4 #include "rclcpp/rclcpp.hpp"
5 #include "rclcpp_lifecycle/lifecycle_node.hpp"
6
7 using namespace std::chrono_literals;
8
9 class PickupAction : public plansys2::ActionExecutorClient
10 {
11 public:
12     PickupAction() : plansys2::ActionExecutorClient("pick-up", 250ms)
13     {
14         progress_ = 0.0f;
15     }

```

```

16
17 private:
18     void do_work() override
19     {
20         if (progress_ < 1.0f) {
21             progress_ += 0.10f;
22             send_feedback(progress_, "picking up package...");
23         } else {
24             finish(true, 1.0f, "pick-up completed");
25             progress_ = 0.0f;
26         }
27     }
28
29     float progress_;
30 };
31
32 int main(int argc, char ** argv)
33 {
34     rclcpp::init(argc, argv);
35     auto node = std::make_shared<PickUpAction>();
36     node->set_parameter(rclcpp::Parameter("action_name", "pick-up"));
37     node->trigger_transition(
38         lifecycle_msgs::msg::Transition::TRANSITION_CONFIGURE);
39     node->trigger_transition(
40         lifecycle_msgs::msg::Transition::TRANSITION_ACTIVATE);
41     rclcpp::spin(node->get_node_base_interface());
42     rclcpp::shutdown();
43     return 0;
44 }

```

Listing 5: Pick-up action executor (src/pick\_up\_action\_node.cpp).

```

1  #include <memory>
2  #include <string>
3  #include "plansys2_executor/ActionExecutorClient.hpp"
4  #include "rclcpp/rclcpp.hpp"
5  #include "rclcpp_lifecycle/lifecycle_node.hpp"
6
7  using namespace std::chrono_literals;
8
9  class DropOffAction : public plansys2::ActionExecutorClient
10 {
11 public:
12     DropOffAction() : plansys2::ActionExecutorClient("drop-off", 250ms)
13     {
14         progress_ = 0.0f;
15     }
16

```

```

17 private:
18     void do_work() override
19     {
20         if (progress_ < 1.0f) {
21             progress_ += 0.10f;
22             send_feedback(progress_, "dropping off package...");
23         } else {
24             finish(true, 1.0f, "drop-off completed");
25             progress_ = 0.0f;
26         }
27     }
28
29     float progress_;
30 };
31
32 int main(int argc, char ** argv)
33 {
34     rclcpp::init(argc, argv);
35     auto node = std::make_shared<DropOffAction>();
36     node->set_parameter(rclcpp::Parameter("action_name", "drop-off"));
37     node->trigger_transition(
38         lifecycle_msgs::msg::Transition::TRANSITION_CONFIGURE);
39     node->trigger_transition(
40         lifecycle_msgs::msg::Transition::TRANSITION_ACTIVATE);
41     rclcpp::spin(node->get_node_base_interface());
42     rclcpp::shutdown();
43     return 0;
44 }

```

Listing 6: Drop-off action executor (src/drop\_off\_action\_node.cpp).

### 3.5.6 CMakeLists.txt

```

1  cmake_minimum_required(VERSION 3.8)
2  project(transport_task)
3
4  find_package(ament_cmake REQUIRED)
5  find_package(rclcpp REQUIRED)
6  find_package(rclcpp_lifecycle REQUIRED)
7  find_package(lifecycle_msgs REQUIRED)
8  find_package(plansys2_executor REQUIRED)
9
10 set(DEPS
11     rclcpp
12     rclcpp_lifecycle
13     lifecycle_msgs
14     plansys2_executor

```



```

15 )
16
17 add_executable(navigate_action_node src/navigate_action_node.cpp)
18 ament_target_dependencies(navigate_action_node ${DEPS})
19
20 add_executable(pick_up_action_node src/pick_up_action_node.cpp)
21 ament_target_dependencies(pick_up_action_node ${DEPS})
22
23 add_executable(drop_off_action_node src/drop_off_action_node.cpp)
24 ament_target_dependencies(drop_off_action_node ${DEPS})
25
26 install(TARGETS
27     navigate_action_node
28     pick_up_action_node
29     drop_off_action_node
30     DESTINATION lib/${PROJECT_NAME}
31 )
32
33 install(DIRECTORY pddl launch config
34     DESTINATION share/${PROJECT_NAME}
35 )
36
37 ament_package()

```

Listing 7: CMakeLists.txt for the transport\_task package.

### 3.5.7 Launch File

```

1 import os
2 from ament_index_python.packages import get_package_share_directory
3 from launch import LaunchDescription
4 from launch_ros.actions import Node
5 from plansys2_support_py.launch import generate_plansys2_launch
6
7 def generate_launch_description():
8     pkg_dir = get_package_share_directory('transport_task')
9     domain_file = os.path.join(pkg_dir, 'pddl', 'transport_domain.pddl')
10
11     plansys2_nodes = generate_plansys2_launch(domain_file=domain_file)
12
13     navigate_node = Node(
14         package='transport_task',
15         executable='navigate_action_node',
16         name='navigate_action_node',
17         output='screen',
18     )
19

```

```

20     pick_up_node = Node(
21         package='transport_task',
22         executable='pick_up_action_node',
23         name='pick_up_action_node',
24         output='screen',
25     )
26
27     drop_off_node = Node(
28         package='transport_task',
29         executable='drop_off_action_node',
30         name='drop_off_action_node',
31         output='screen',
32     )
33
34     return LaunchDescription(
35         plansys2_nodes + [navigate_node, pick_up_node, drop_off_node]
36     )

```

Listing 8: Python launch file that starts PlanSys2 core nodes, loads the PDDL domain, and launches the three action executors (`launch/transport_task.launch.py`).

### 3.5.8 Building and Running

After placing the package in a ROS2 workspace (`colcon workspace`), the build and activation sequence follows the standard ROS2 workflow. First, the workspace is built with `colcon build -symlink-install`, and the environment is sourced with `source install/setup.bash`. The system is then launched with `ros2 launch transport_task transport_task.launch.py`. Once running, the problem instance (objects, predicates, and goal) is loaded into the Problem Manager via the PlanSys2 terminal client or through programmatic ROS2 service calls, and planning is triggered.

### 3.5.9 Expected Execution Behavior

Figure 4 illustrates the expected action sequence produced by the POPF solver and dispatched by the PlanSys2 Executor for this problem instance. Because the robot can only carry one package at a time (**free** predicate prevents concurrent pick-up), the plan serializes the two delivery sub-tasks.

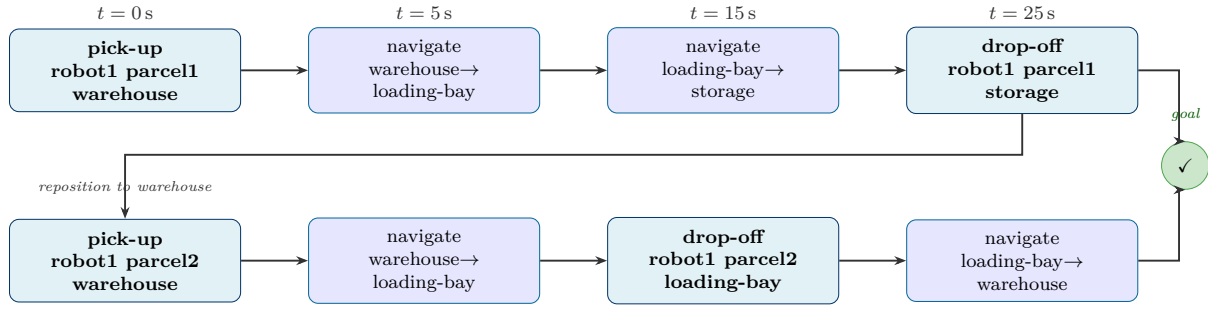


Figure 4: Expected plan execution sequence for the transport task. Row 1 handles parcel1 (deliver to storage); Row 2 handles parcel2 (deliver to loading-bay, return robot home). Durations: pick-up/drop-off = 5s; navigate = 10s. The tick mark indicates goal achievement.

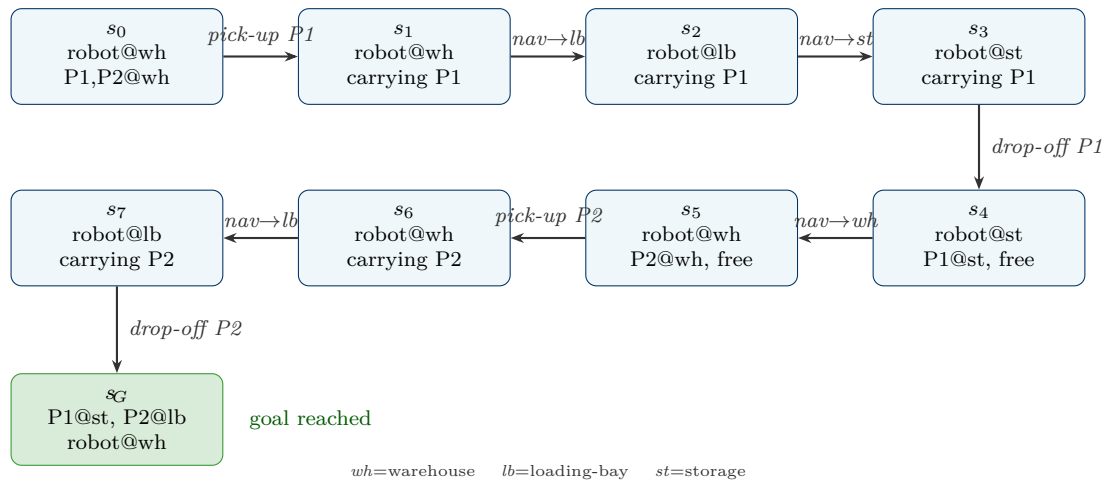


Figure 5: World state sequence corresponding to the transport plan. Each state transition corresponds to one PDDL action execution. The path snakes: top row left-to-right, then bottom row right-to-left, ending at the goal state  $s_G$ .

### 3.6 Known Limitations of PlanSys2

Despite its practical utility, PlanSys2 exhibits limitations that are directly relevant to this research:

**Epistemic limitations.** PlanSys2 is fundamentally a classical single-agent planning model. The world state is a set of ground predicates assumed to be fully known to the planner. There is no representation of agent-relative knowledge, no epistemic goal language, and no support for actions that change knowledge states without changing the physical world.

**Partial observability.** PlanSys2 does not natively handle partial observability. Sensor uncertainty and missing observations must be handled outside the planner before updating predicates. The planner always assumes predicates reflect the true world state.

**Multi-agent coordination.** PlanSys2 is designed for single-robot planning. While multiple instances can run in parallel, there is no native mechanism for cross-robot plan coordination, shared world state, or multi-agent goal management.

**Solver expressivity.** The solvers supported by PlanSys2 (POPF, TFD) implement classical or temporal planning with no support for epistemic or probabilistic reasoning. Extending PlanSys2 to use an epistemic planner would require a new solver plugin and a restructured problem representation.

These limitations motivate the core research objective: to integrate the engineering infrastructure of ROS2 and PlanSys2 with the expressive semantics of DEL-based epistemic planning.

## 4 Multi-Robot Systems in ROS2

---

### 4.1 Architectural Patterns for Multi-Robot Systems

Multi-robot systems in ROS2 are typically organized according to one of three architectural patterns: *centralized*, *decentralized*, or *hierarchical*. In a centralized architecture, a single master node maintains a global world model and generates plans for all robots. In a decentralized architecture, each robot operates as an independent planning agent, communicating with peers to resolve conflicts and share observations. A hierarchical architecture arguably the most practically relevant for the target research combines a fleet-level planner with robot-level planners, where the fleet planner decomposes high-level goals into sub-goals assigned to individual robots.

### 4.2 Multi-Robot Navigation and Coordination

Navigation in multi-robot ROS2 systems is typically managed by the Nav2 stack, supporting namespaced multi-robot deployments. Open-RMF (Open Robotics Middleware Framework) provides a standardized interface for multi-fleet, multi-robot task allocation and coordination in shared environments [9].

### 4.3 Task Allocation and Multi-Agent Planning

Task allocation assigns tasks to robots to optimize objectives such as makespan, coverage, or load balance. Classical approaches include market-based auction methods and integer programming formulations. In the context of PlanSys2, one approach is to model multiple robot agents as PDDL objects in a single planning instance, leveraging MA-PDDL or MASTRIPS extensions. However, such approaches remain within the classical planning paradigm and inherit all its epistemic limitations.

### 4.4 Shared World Modeling

A fundamental challenge is maintaining a shared, consistent world model across all agents. From an epistemic planning perspective, the problem is more subtle: it is not always desirable or possible to maintain a single globally consistent model. Different robots may have genuinely different beliefs about the state of the world, and those differences are semantically meaningful. An epistemic planner must represent and reason about individual perspectives, plan actions that resolve epistemic differences, and formulate goals referring explicitly to agent knowledge states.

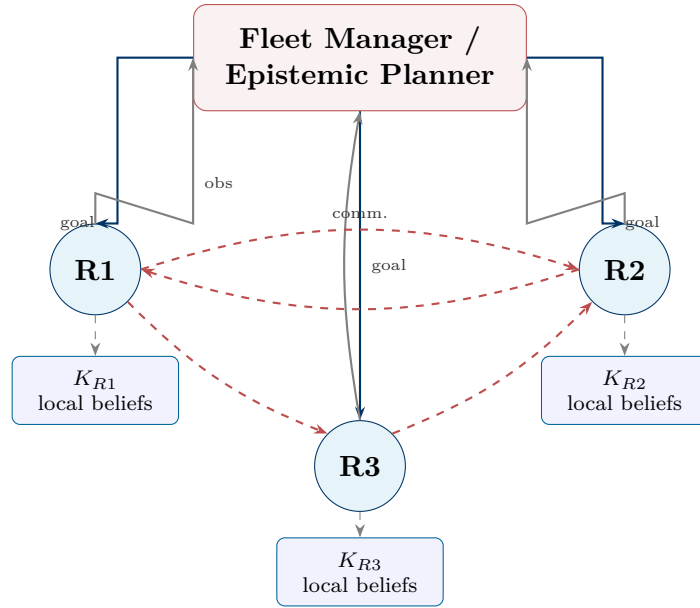


Figure 6: Multi-robot system with a fleet-level epistemic planner and robot-level local knowledge bases. Robots send observations to the fleet planner and engage in peer-to-peer communication, giving rise to heterogeneous epistemic states.

## 5 Dynamic Epistemic Logic: Foundations for Epistemic Planning

### 5.1 Motivation: From Classical to Epistemic Planning

Classical planning operates over a deterministic, fully observable world. It cannot represent the knowledge states of individual agents, uncertainty about the current state, or the epistemic effects of communication. Dynamic Epistemic Logic (DEL) [3, 4] provides a rigorous formal framework for reasoning about knowledge, belief, and informational change in multi-agent systems, extending classical propositional logic with modal operators and a formal semantics for how truth values change when epistemic actions are applied.

### 5.2 Epistemic States and Kripke Semantics

**Definition 5.1** (Multi-Agent Epistemic Model). A multi-agent epistemic model is a triple  $M = (W, R, L)$  where:

- $W \neq \emptyset$  is a finite set of *possible worlds*;
- $R : \text{Ag} \rightarrow 2^{W \times W}$  assigns to each agent  $i$  an *accessibility relation*  $R_i \subseteq W \times W$ ; and
- $L : W \rightarrow 2^P$  assigns to each world a *label* the set of atoms true there.

An epistemic state is a pair  $s = (M, W_d)$  where  $W_d \subseteq W$  is a non-empty set of *designated worlds*.

The accessibility relation  $wR_iv$  means that in world  $w$ , agent  $i$  considers  $v$  possible given its current observations. A formula  $\Box_i\varphi$  (“agent  $i$  knows  $\varphi$ ”) holds in  $w$  iff  $\varphi$  holds in all  $R_i$ -accessible worlds:

$$(M, w) \models \Box_i\varphi \iff \forall v \in W : wR_iv \Rightarrow (M, v) \models \varphi$$

The *knowing-whether* modality  $\text{Kw}_i\varphi = \Box_i\varphi \vee \Box_i\neg\varphi$  is especially important in robotics, where sensors resolve binary queries.

### 5.3 Common Knowledge

For a group  $G \subseteq \text{Ag}$ , *common knowledge*  $C_G\varphi$  requires that all agents know  $\varphi$ , all agents know that all know it, and so on ad infinitum:

$$C_G\varphi = \bigwedge_{k>0} \Box_G^k\varphi, \quad \Box_G\varphi = \bigwedge_{i \in G} \Box_i\varphi$$

Common knowledge is essential for robotic coordination: a team that must act in concert requires not only that each robot knows the plan, but that it is common knowledge among all robots that they know the plan otherwise a robot might fail to act because it is

uncertain whether its teammates are aware of their roles.

#### 5.4 Epistemic Actions and the Product Update

**Definition 5.2** (Event Model and Epistemic Action). An event model is a quadruple  $A = (E, Q, \text{pre}, \text{post})$  where  $E \neq \emptyset$  is a finite set of *events*;  $Q : \text{Ag} \rightarrow 2^{E \times E}$  assigns each agent an accessibility relation over events;  $\text{pre} : E \rightarrow \mathcal{L}_{P, \text{Ag}}^C$  assigns preconditions; and  $\text{post} : E \times P \rightarrow \mathcal{L}_{P, \text{Ag}}^C$  assigns postconditions. An epistemic action is a pair  $a = (A, E_d)$  where  $E_d \subseteq E$  are the *designated events*.

**Definition 5.3** (Product Update). Let  $a = ((E, Q, \text{pre}, \text{post}), E_d)$  be applicable in state  $s = ((W, R, L), W_d)$ . The product update  $s \otimes a = ((W', R', L'), W'_d)$  is:

$$\begin{aligned} W' &= \{(w, e) \in W \times E \mid (M, w) \models \text{pre}(e)\} \\ R'_i &= \{((w, e), (v, f)) \mid w R_i v \text{ and } e Q_i f\} \\ L'(w, e) &= \{p \in P \mid (M, w) \models \text{post}(e, p)\} \\ W'_d &= \{(w, e) \in W' \mid w \in W_d, e \in E_d\} \end{aligned}$$

The product update simultaneously captures physical effects (through postconditions) and epistemic effects (through combined world and event accessibility relations). An agent's new beliefs are determined by what it knew before combined with what it can observe about the action.

#### 5.5 Types of Epistemic Actions

A *public announcement*  $\text{ann}(\varphi)$  communicates a formula to all agents simultaneously with a reflexive accessibility relation. It is the simplest DEL action and forms the basis of the Public Announcement Logic (PAL) fragment.

A *private ontic action* is one where the actor moves a physical object while other agents are oblivious modeled with a “null event” linked to the real event via non-actor accessibility.

A *sensing action* allows agents to observe a binary property, with two designated events for each outcome. After sensing, a fully observant agent knows whether the property holds.

A *quasi-private sensing action* combines private and semi-private observability: some agents are fully observant, some know only that sensing occurred, and some are oblivious.

#### 5.6 Epistemic Planning Tasks

**Definition 5.4** (Epistemic Planning Task). An epistemic planning task is a triple  $T = (s_0, \text{Act}, \varphi_g)$  where  $s_0$  is an initial epistemic state,  $\text{Act}$  is a finite set of epistemic actions, and  $\varphi_g$  is a goal formula. A solution is a sequence  $\pi = a_1, \dots, a_l$  such that  $\pi$  is applicable



in  $s_0$  and  $s_0 \otimes \pi \models \varphi_g$ .

Crucially,  $\varphi_g$  may be an epistemic formula: for example,  $\Box_{R_1} \text{packageDelivered} \wedge K_{R_2} \text{corridorBlocked}$  specifies knowledge states as goals not expressible in classical PDDL.

## 5.7 DEL in Robotics: From Theory to Application

The transfer of DEL from its origins in philosophical logic and game theory to practical multi-robot systems has been one of the most active research directions of the past decade. While the theoretical framework of DEL was originally developed for studying abstract epistemic puzzles [3, 4], the modeling machinery it provides Kripke models, accessibility relations, product updates maps naturally onto the challenge of reasoning about a distributed robot fleet in which agents have heterogeneous, potentially inconsistent observations of a shared environment.

The foundational insight is that a robot’s *epistemic state* is not merely a record of what it has sensed, but a structured representation of what it knows, what it does not know, and crucially in a multi-robot setting what it knows or believes about the epistemic states of its teammates. This higher-order structure, formalized through nested modal operators  $\Box_i \Box_j \varphi$  and common knowledge  $C_G \varphi$ , has no counterpart in classical PDDL-based planning and therefore cannot be captured by conventional ROS2 task planners such as PlanSys2.

An early and influential formalization of DEL for autonomous agents is provided by Löwe, Pacuit, and Witzel [18], who define the *DEL planning problem* as the task of finding a sequence of event models  $\sigma$  such that  $M \otimes \sigma \models \varphi_g$ , where  $(M, w)$  is the initial pointed model and  $\varphi_g$  is an epistemic goal formula. Their analysis of the *Thief: The Dark Project* scenario demonstrates that a knowledge module maintaining the Kripke model throughout the execution of a plan can support rich, belief-aware scripted and autonomous behaviors without requiring explicit re-engineering of the agent’s control logic every time epistemic assumptions change. They also establish structural properties *self-absorption* and *commutativity* of propositional almost-mutex event models that bound the growth of the Kripke model under iterated updates. Concretely, for the event library  $\mathcal{E} = \{N_t, N_g, B_t, B_g, F\}$  (noise and facing actions in the stealth game scenario), the bisimulation contraction of  $I \otimes \sigma$  has size at most 6 for any legal sequence  $\sigma$ , regardless of length. This is an important tractability result: it shows that for suitably restricted event libraries, the Kripke model does not grow without bound and can be maintained in real time.

The framework of Löwe et al. also formalizes three planning problem variants of increasing difficulty. In the *absolute* problem, no agent assignment is specified and the planner searches for any legal sequence achieving the goal. In the *single-agent* problem, all events must lie within the power set  $\text{power}(i)$  of a designated agent  $i$ . In the *adversarial* problem

most relevant to adversarial environments or unreliable communication the planner must find a partial strategy  $\hat{\sigma}$  such that the goal is achieved under *all* legal completions, forming a game-theoretic worst-case guarantee. These distinctions directly inform the design of epistemic planning modules for robotic systems, where the choice between cooperative and adversarial problem formulations determines the computational complexity of planning and the robustness guarantees that can be provided.

## 5.8 Belief and Empathy States in Multi-Robot DEL

A central design choice in applying DEL to robotics is how to represent epistemic states at runtime in a way that is both formally grounded and computationally tractable. Bramblett and Bezzo [16] propose an elegant particle-based approximation in which each robot  $i$  maintains a finite set of *belief particles*  $\mathcal{P}_i = \{p_{ij,k} \mid j \in \mathcal{A}, k \in \mathcal{A}\}$ , where  $p_{ij,k}$  represents robot  $i$ 's second-order belief its belief about robot  $j$ 's belief about robot  $k$ 's state. The designated subset  $C_i \subseteq \mathcal{P}_i$ , called the *common belief set*, corresponds to the last globally shared epistemic state; it is propagated forward in time using each robot's known dynamics and status propositions when communication is unavailable.

This formulation instantiates the DEL epistemic state  $(W, R_i, V, W_d)$  with a finite, tractable world set: worlds  $w \in W$  are indexed by combinations of possible robot statuses  $\sigma_i^t \in \{\text{exploring, gossiping, performing task, going home}\}$  for all  $i \in \mathcal{A}$ . The accessibility relation  $R_i$  then encodes robot  $i$ 's uncertainty about which status combination is currently true. Epistemic updates follow from two action types in the library  $\mathcal{A} = \{\text{perceive}(\varphi), \text{announce}(\varphi)\}$ : a *perceive* action updates a robot's local belief when it directly observes a proposition  $\varphi$  (e.g., task discovered or teammate absent from expected location), and an *announce* action a DEL public announcement disseminates a state description to all locally connected robots, establishing common knowledge of the announced formula among them.

The product update semantics govern rational belief revision in each case. When all robots are connected and robot  $i$  announces the full system disposition  $\Omega = (\Omega_j)_{j \in \mathcal{A}}$ , the epistemic state from robot  $i$ 's perspective evolves as:

$$s_{t-1}^i \otimes \text{announce}(\Omega) = s_t^i \models K_i \sigma_t^i \wedge \bigwedge_{j \in \mathcal{A}} K_i K_j \sigma_t^j,$$

establishing mutual knowledge of all robots' statuses the condition for a true world  $w_t^*$  to exist in the Kripke model. When robots are disconnected, the perceive action produces a local, asymmetric belief update: if robot  $i$  observes that robot  $j$  is absent from its expected location, the world  $w$  corresponding to  $j$ 's originally predicted status is removed from  $R_i$ 's accessible set, and the next most probable particle is activated. This selective pruning of the world set corresponds precisely to the DEL product update in the special case where

the event model has two events “ $j$  is here” and “ $j$  is not here” and the precondition of the former is not satisfied.

The companion paper by the same authors [15] extends this framework to heterogeneous robot teams (mixed UGV/UAV fleets) and introduces the concept of *empathy states*: a robot  $i$  maintains, alongside its belief particles about others, a set of particles representing what other robots might currently believe about  $i$  itself. Formally,  $\mathcal{P}_i^e = \{p_{ii,b} \mid b \in B\}$  are robot  $i$ ’s empathy particles, and they are equivalent to robot  $j$ ’s belief particles about robot  $i$  under the assumption that all robots follow the same particle-propagation policy. This symmetry property enables a robot to proactively position itself so that at least one of its teammates’ belief particles correctly predicts its location, ensuring future communicability without explicit signaling.

The epistemic task allocation problem in this setting is formalized as a non-linear integer program over the epistemic state  $s_t^i$ , where each robot  $i$  optimizes a joint utility function incorporating expected arrival times computed from the belief particles, capability constraints, and precedence constraints over task orderings. The solution, computed via a genetic algorithm with the initial population seeded by a greedy feasible-solution generator, produces a policy  $\pi$  as a concatenation of gossip bundles (which robot communicates with whom) and task bundles (who performs what). A gossip protocol then propagates task assignments through the network of disconnected robots, exploiting the particle-based belief model to route messages efficiently through predicted teammate locations [15, 16].

Simulation experiments over 15 randomized environments with two to eight robots demonstrated that the proposed DEL-based method achieves mission times close to an ideal fully-connected baseline and substantially outperforms a connectivity-constrained flock method across all team sizes. This provides strong empirical evidence that the particle-based DEL approximation is not merely a theoretical tool but a practically viable approach to runtime epistemic reasoning in robotic systems.

## 5.9 Theory of Mind, Higher-Order Reasoning, and Active Epistemic Inference

A significant limitation of first-order epistemic planning where each robot reasons only about its own beliefs about the world is that it cannot resolve ambiguities arising from simultaneous actions of multiple agents without communication. If two robots observe each other moving toward the same task, neither can determine from its own first-order observations alone who is actually committed to that task, leading to convergence failures and duplicated effort. The epistemic solution is to elevate the depth of reasoning to second and third order: a robot  $i$  reasons not only about what others know ( $\Box_j \varphi$ ) but about what others know about what  $i$  knows ( $\Box_j \Box_i \varphi$ ), capturing the “I know that you know that I know” structure of Theory of Mind (ToM) [17].

Bramblett, Reasoner, and Bezzo formalize this in an *active epistemic inference* framework that combines DEL-based higher-order belief maintenance with variational free energy minimization from active inference theory [17]. The epistemic state is represented as  $s = (W, (R_i)_{i \in \mathcal{R}}, L, w)$ , where the accessibility relation  $R_i$  encodes robot  $i$ 's run-time uncertainty over world configurations each world  $w \in W$  being an assignment of robots to task subsets. Composite accessibility relations  $R_i \circ R_j$  encode second-order knowledge:  $s \models K_i K_j \varphi$  iff  $\varphi$  holds in all worlds jointly reachable through both  $R_i$  and  $R_j$ . Plans for each robot are sequences  $\pi_i = \langle a_{i1}, \dots, a_{ip} \rangle$  leading from an initial epistemic state  $s_i^0$  to a goal state  $s_i^* \in \Gamma_i$ , where the planning function incorporates the nested belief structure:  $\pi_i = \text{Plan}(s_i^0, \Gamma_i, \mathbf{B}_i)$ , with  $\mathbf{B}_i$  encoding robot  $i$ 's beliefs about the beliefs of all other robots.

The active inference layer converts epistemic beliefs into motion policies by minimizing the *variational free energy* (VFE):

$$\mathcal{F}(x_i(t), u_i(t), \mathcal{G}, y_i(t), \omega_i) = H[q(\mathcal{G})] + D_{\text{KL}}[q(\mathcal{G}) \parallel P(x_i(t) \mid y_i(t), \omega_i)],$$

where  $q(\mathcal{G})$  is the robot's approximate posterior over valid task-assignment configurations  $\mathcal{G}$ , and  $\omega_i$  is its sensor configuration. The likelihood function that drives belief updates is computed using a hierarchical *salience* function indexed by reasoning order  $k$ :

$$e_i^{(k)} \leftarrow v_i^{(k)}(y_i, \Omega, \mathcal{G}) = \exp\left(-\frac{1}{\eta} h_i^{(k)}(y_i, \Omega, \mathcal{G})\right),$$

where zero- and first-order evidence use each robot's own direct observations; second-order evidence abstracts the perspective of another robot by using its known sensor model  $\omega_j$ ; and third-order evidence models what robot  $j$  believes robot  $k$  observes. The joint likelihood over all orders is shown to reduce the variance of the posterior belief distribution compared to first-order reasoning alone, by averaging out bounded, mean-zero observation errors across multiple reasoning layers [17].

In practice, robots execute a *perception-policy loop*: update the posterior  $q(\mathcal{G})$  from observations, compute the expected free energy for a set of candidate movement policies, and select the policy minimizing EFE. Policies that actively disambiguate goal assignments by taking “exaggerated” movements that are informative to observers are preferred over purely task-directed paths because they minimize the epistemic term of the EFE. This produces implicit coordination: robots signal intent through motion without any explicit communication, resolving assignment conflicts that would stalemate first-order reasoners. Experiments with Husarion ROSbot 2.0 UGVs and Crazyflie UAVs confirm that higher-order reasoning (up to third order) achieves substantially higher mission success rates across rendezvous, task allocation, and multi-task scenarios, with the advantage growing markedly as team size exceeds three robots [17].

### 5.10 DEL Planning Problem Variants and Structural Properties

The formal study of DEL-based planning admits several natural variants depending on the degree of agent autonomy and the nature of the environment. Following the framework of Löwe et al. [18], one distinguishes:

- **Absolute DEL planning:** Find a sequence  $\sigma \in T \subseteq LS$  of event models such that  $M \otimes \sigma \models \varphi$ , where  $T$  is a subtree of legal sequences and  $\varphi$  is an epistemic goal formula. This formulation is agent-agnostic.
- **Single-agent DEL planning:** As above, but additionally requiring  $\sigma \in \text{power}(i)$ , where the function  $\text{power} : \mathcal{A} \rightarrow \wp(\mathcal{E})$  maps each agent to the set of event models it can instantiate. This corresponds to the classical planning assumption of a single omniscient planner.
- **Adversarial DEL planning:** Find a partial strategy function  $\hat{\sigma} : \{0, \dots, N\} \rightarrow \mathcal{E}$  such that (i)  $\hat{\sigma} \in \text{power}(i)$ , (ii) there exists a completion  $\sigma \in T$  with  $\hat{\sigma} \subseteq \sigma$ , and (iii) for *all* such completions  $M \otimes \sigma \models \varphi$ . This is a worst-case game-theoretic problem in which the planning agent must achieve its epistemic goal regardless of the actions of opponents or environmental nondeterminism.

The adversarial formulation is the most directly relevant to multi-robot systems operating with unreliable communication channels or nondeterministic physical outcomes. Critically, the goal formula  $\varphi$  in all three variants is drawn from the full epistemic language and may therefore specify goals such as  $\Box_{R_1} \text{taskComplete} \wedge C_A \text{rendezvousAgreed}$  that cannot be reduced to classical propositional planning goals.

Two structural properties identified by Löwe et al. are directly relevant to the runtime tractability of DEL planning in robotics. An event model  $\mathcal{E}$  is *self-absorbing* if  $M \otimes \mathcal{E} \otimes \mathcal{E} \simeq M \otimes \mathcal{E}$  for all models  $M$  (applying the same event twice has the same effect as applying it once). Two event models  $\mathcal{E}, \mathcal{E}'$  are *commutative* if  $M \otimes \mathcal{E} \otimes \mathcal{E}' \simeq M \otimes \mathcal{E}' \otimes \mathcal{E}$  for all  $M$ . Propositional event models are always commutative (Lemma 1 in [18]), and almost-mutex event models with transitive accessibility relations are self-absorbing (Lemma 2 in [18]). When the event library  $\mathcal{E}$  consists entirely of such event models, the Kripke model remains bounded in size under any sequence of updates, and the bisimulation contraction can be computed in linear time. This guarantees that the World Model Manager in the proposed architecture (Section 7) remains computationally tractable for suitably restricted event libraries a property that should be treated as a design criterion when specifying EPDDL action schemas for concrete robotic domains.

## 6 EPDDL: The Epistemic Planning Domain Definition Language

### 6.1 Motivation and Design Philosophy

EPDDL, introduced by Burigana and Fabiano [5] as the official language for the Epistemic Planning Track at IPC 2026, addresses the absence of a standardized language for DEL-based epistemic planning problems. It is designed for: (1) compatibility with PDDL syntax; (2) formal grounding in full DEL semantics with abstract event models; and (3) a modular action type library system enabling specification and comparison of DEL fragments.

### 6.2 Language Structure

An EPDDL specification consists of three components: **problems** (instance-specific: objects, agents, initial epistemic state, goal); **domains** (instance-independent: type declarations, predicates, event definitions, action schemas); and **action type libraries** (abstract frames defining observability patterns for different DEL fragments).

### 6.3 EPDDL Syntax: A Concrete Example

We present a simplified domain and problem for a two-robot corridor-sensing scenario: robots **r1** and **r2** are adjacent to corridor **c1**, whose blocked status is initially unknown to both. Robot **r1** can sense the corridor and then announce its finding. The goal is that all agents know whether the corridor is blocked.

#### 6.3.1 EPDDL Domain

```

1 (define (domain corridor-sensing)
2   (:requirements :typing :sensing :announcements :group-modalities)
3
4   (:types robot corridor)
5
6   (:predicates
7     (blocked ?c - corridor)
8     (adjacent ?r - robot ?c - corridor)
9   )
10
11   (:action sense-corridor
12     :type sensing
13     :actor ?r - robot
14     :parameters (?r - robot ?c - corridor)
15     :observability (
16       (full-obs ?r)
17       (no-obs All \ {?r})
18     )
19     :precondition (adjacent ?r ?c)

```

```

20   :sense-formula (blocked ?c)
21 )
22
23 (:action announce-blocked
24   :type public-announcement
25   :actor ?r - robot
26   :parameters (?r - robot ?c - corridor)
27   :precondition (and
28     (adjacent ?r ?c)
29     ([?r] (blocked ?c))
30   )
31   :announced-formula (blocked ?c)
32 )
33
34 (:action announce-clear
35   :type public-announcement
36   :actor ?r - robot
37   :parameters (?r - robot ?c - corridor)
38   :precondition (and
39     (adjacent ?r ?c)
40     ([?r] (not (blocked ?c)))
41   )
42   :announced-formula (not (blocked ?c))
43 )
44 )

```

Listing 9: EPDDL domain for a two-robot sensing and announcement scenario (corridor\_sensing.epddl).

### 6.3.2 EPDDL Problem

```

1 (define (problem corridor-task-1)
2   (:domain corridor-sensing)
3
4   (:objects
5     r1 r2 - robot
6     c1 - corridor
7   )
8
9   (:agents r1 r2)
10
11  (:init
12    (:static
13      (adjacent r1 c1)
14      (adjacent r2 c1)
15    )
16    (:epistemic

```



```

17      (C All (Kw r1 (blocked c1)) false)
18      (C All (Kw r2 (blocked c1)) false)
19    )
20  )
21
22  (:goal (and
23    ([Kw. All] (blocked c1))
24  ))
25 )

```

Listing 10: EPDDL problem instance using a finitary S5-theory for the initial epistemic state (`corridor_task.epddl`).

The `:epistemic` block encodes a finitary S5-theory: `(C All (Kw r1 (blocked c1)) false)` asserts that it is common knowledge among all agents that `r1` does not know whether the corridor is blocked. The goal `([Kw. All] (blocked c1))` uses the group knowing-whether modality to require all agents to resolve the uncertainty. Key syntactic constructs are summarized in Table 1.

| EPDDL Construct                        | Logical Meaning                                   | Context              |
|----------------------------------------|---------------------------------------------------|----------------------|
| <code>([i] phi)</code>                 | $\Box_i \varphi$ agent $i$ knows $\varphi$        | preconditions, goals |
| <code>(&lt;i&gt; phi)</code>           | $\Diamond_i \varphi$ agent $i$ considers possible | preconditions        |
| <code>([Kw. i] phi)</code>             | $Kw_i \varphi$ knows whether $\varphi$            | goals, init          |
| <code>([Kw. All] phi)</code>           | $Kw_{All} \varphi$ all know whether               | goals                |
| <code>(C All phi)</code>               | $C_{All} \varphi$ common knowledge                | init, goals          |
| <code>:type sensing</code>             | sensing action type                               | action schema        |
| <code>:type public-announcement</code> | public announcement type                          | action schema        |
| <code>:sense-formula</code>            | formula whose truth is observed                   | sensing actions      |
| <code>:announced-formula</code>        | formula publicly communicated                     | announcement actions |
| <code>:observability</code>            | per-type observability assignment                 | action schema        |

Table 1: Key EPDDL syntactic constructs with logical counterparts and usage contexts.

## 6.4 Modal Operators in EPDDL

| EPDDL Syntax                     | Logical Formula          | Reading                                   |
|----------------------------------|--------------------------|-------------------------------------------|
| <code>([i] phi)</code>           | $\Box_i \varphi$         | Agent $i$ knows/believes $\varphi$        |
| <code>(&lt;i&gt; phi)</code>     | $\Diamond_i \varphi$     | Agent $i$ considers $\varphi$ possible    |
| <code>([Kw. i] phi)</code>       | $Kw_i \varphi$           | Agent $i$ knows whether $\varphi$         |
| <code>(&lt;Kw. i&gt; phi)</code> | $\widehat{Kw_i \varphi}$ | Agent $i$ does not know whether $\varphi$ |
| <code>([C. G] phi)</code>        | $C_G \varphi$            | $\varphi$ is common knowledge in $G$      |
| <code>(&lt;C. G&gt; phi)</code>  | $\hat{C}_G \varphi$      | Dual of common knowledge                  |

Table 2: Modal operators in EPDDL and their logical counterparts.



Modality indices may be single agent terms, agent group lists, or the reserved keyword All. Group modalities require the `:group-modalities` requirement flag.

## 6.5 Initial State Representation

EPDDL supports two methods for specifying the initial epistemic state. An *explicit definition* directly encodes worlds, accessibility relations, world labels, and designated worlds. A *finitary S5-theory* [11] specifies the state through commonly-known epistemic formulas of restricted form, consisting of propositional atoms describing the actual world,  $C_{Ag}(\Box_i \varphi)$  formulas for commonly-known truths,  $C_{Ag}(Kw_i \varphi)$  formulas for agent certainties, and  $C_{Ag}(\widehat{Kw}_i \varphi)$  formulas for agent uncertainties. The **plank** toolkit automatically constructs the Kripke model from a finitary S5-theory.

## 6.6 DEL Fragments via Action Type Libraries

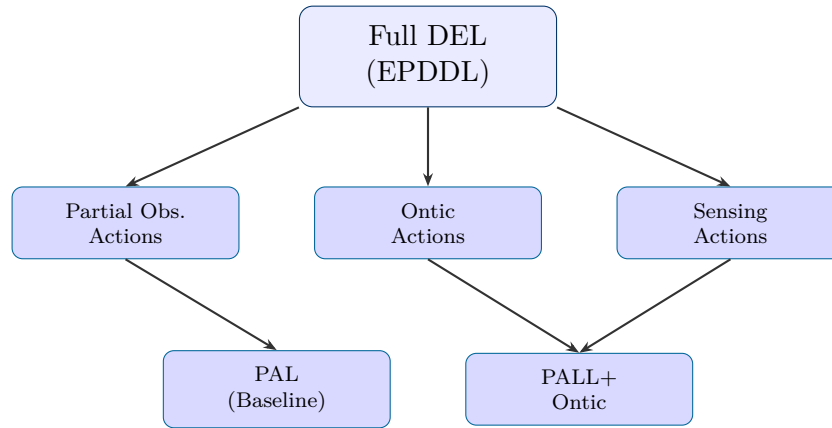


Figure 7: Hierarchy of DEL fragments expressible through EPDDL action type libraries. The PAL baseline supports only public announcements. Richer fragments add ontic actions, partial observability, and sensing actions.

Action type libraries allow benchmark designers to create problem sets solvable by planners of different capabilities. Any epistemic planning formalism translatable into a DEL fragment can be represented in EPDDL via an appropriate library, enabling cross-formalism comparison.

## 6.7 Abstract Event Models and Action Types

The key theoretical innovation is the *abstract epistemic action*, which decouples the structural pattern of observability (action type) from specific preconditions, effects, and agent assignments (domain action).

**Definition 6.1** (Abstract Event Model). An abstract event model on ObsTypes is a tuple  $(E, Q, \text{pre}, \text{post}, \text{obs})$  where  $(E, Q)$  is an abstract frame, pre and post are as in a standard

event model, and  $\text{obs} : \text{Ag} \rightarrow (\text{ObsTypes} \rightarrow \mathcal{L}_{P,\text{Ag}}^C)$  assigns to each agent an observability function satisfying mutual inconsistency and coverage of the logical space.

## 7 Integration: Toward a DEL-Based Planning System for ROS2

### 7.1 Conceptual Architecture

Integrating DEL-based epistemic planning with a ROS2 multi-robot system requires bridging three conceptual gaps: (i) between the physical robot state and the epistemic state representation; (ii) between EPDDL action schemas and ROS2 action server implementations; and (iii) between the planner’s output a sequence of abstract epistemic actions and the robot execution infrastructure.

We propose a layered conceptual architecture composed of three interacting strata: a *Cognitive Layer*, an *Execution Layer*, and a *Perception Layer*. This separation clarifies the interaction between symbolic epistemic reasoning and physical multi-robot control.

**Cognitive Layer.** This layer maintains and manipulates the global epistemic state  $(M, W_d)$  of the fleet. It contains:

- **Epistemic World Model Manager:** Maintains the Kripke model and applies product update semantics (or suitable approximations) upon perception events or action completion. In the spirit of the knowledge module proposed by Löwe et al. [18], this component acts as a persistent epistemic engine that any behavior module can query for truth values of epistemic formulas.
- **EPDDL Planner Interface:** Generates EPDDL problem instances from the current epistemic state, invokes an external epistemic planner, and translates the resulting abstract plan into dispatchable actions.
- **Epistemic Action Dispatcher:** Instantiates abstract epistemic actions as physical or communication events, dispatches them to ROS2, and triggers the corresponding epistemic product update.

**Execution Layer.** Implements low-level robot behaviors using standard ROS2 action servers (navigation, manipulation, sensing, communication). This layer is responsible for physical actuation and execution monitoring.

**Perception Layer.** Transforms physical sensor data into symbolic assertions and epistemic events. A state estimation and belief update module translates observations into grounded propositions and applies updates to the global epistemic state.

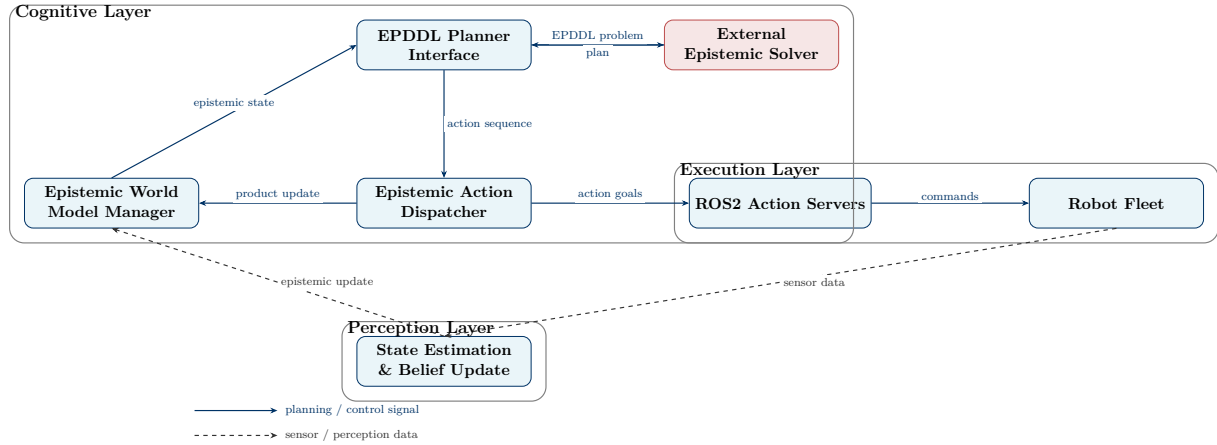


Figure 8: Layered conceptual architecture for DEL-based epistemic multi-robot planning. The Cognitive Layer maintains and updates the global Kripke model and generates epistemic plans. The Execution Layer implements physical action through ROS2 action servers. The Perception Layer translates sensor observations into symbolic epistemic updates applied via DEL product update semantics.

## 7.2 State Space and Computational Challenges

The number of possible worlds in a Kripke model can grow exponentially with the number of propositions and agents. Several approximation strategies have been explored: *bounded world representations* limit the number of worlds using heuristic pruning; the particle-based approximation of Bramblett and Bezzo [16] offers a computationally efficient alternative that preserves the essential DEL update structure while bounding world-set size; and *compiled classical planning approaches* [10] transform epistemic problems into classical ones by encoding epistemic state as additional propositional atoms. The structural tractability results of Löwe et al. [18] for self-absorbing and commutative event libraries provide a formal basis for bounding model growth and should inform the choice of EPDDL action schemas in a deployed system.

## 7.3 Perception-to-Epistemic-State Translation

A critical engineering challenge is translating robot sensor data into Kripke model updates. A robot’s observation a camera image, a lidar scan is a physical signal that must be interpreted as an epistemic event. In the ROS2 context, this pipeline consists of a perception node (producing symbolic object detections or map assertions), a semantic interpretation layer (translating detections into PDDL-compatible ground atoms), and an epistemic update module (applying the product update to the World Model Manager’s Kripke model).

## 7.4 Communication Actions as Epistemic Events

Inter-robot communication is itself an epistemic event: when robot  $R_1$  broadcasts that corridor  $c_3$  is blocked, this constitutes a public announcement of  $\Box_{R_1} \text{corridorBlocked}(c_3)$ , and the epistemic state changes via the DEL product update. In the applied systems of Bramblett et al. [15, 16], this is implemented via a gossip protocol in which robots route to the predicted locations of teammates and deliver task assignments, treating each successful rendezvous as a localized public announcement that updates both robots' belief states. In the proposed architecture, the Epistemic Action Dispatcher implements communication actions as ROS2 topic publications while simultaneously applying the corresponding product update to the World Model Manager's Kripke model.

## 7.5 Open Research Questions

**Tractable epistemic state representations.** What Kripke model representations are both expressive enough for EPDDL-level reasoning and compact enough for real-time multi-robot fleets? Are factored or compiled representations sufficient? The particle-based approach of [16] and the bisimulation bounding results of [18] offer complementary partial answers.

**Online epistemic planning.** What are the performance characteristics of available epistemic planners (EFP, mGPT, MCEPlan) in online replanning settings relevant to robotics?

**EPDDL-to-ROS2 toolchain.** How should the `plank` toolkit integrate with the ROS2 build and runtime systems? Can EPDDL problem files be auto-generated from ROS2 system state?

**Higher-order reasoning at scale.** The active epistemic inference results of [17] show that third-order reasoning significantly improves task allocation success rates, but the computational cost grows with the number of reasoning levels and robots. What is the optimal trade-off between reasoning depth and computational budget?

**Failure handling.** When epistemic plan actions fail, the epistemic state must be updated and a new plan generated. What recovery strategies are appropriate in a ROS2 runtime?

## 8 Conclusion

---

This document has surveyed the two pillars of a planned research program: the engineering infrastructure of ROS2 and PlanSys2 for task-level robot planning, and the theoretical framework of Dynamic Epistemic Logic and EPDDL for epistemic multi-agent planning. The limitations of classical PDDL-based planners no representation of agent knowledge, no epistemic goal language, no multi-agent coordination semantics motivate the adoption of DEL as the formal backbone of a next-generation planning system for heterogeneous robot fleets.

Recent applied work has demonstrated the practical viability of DEL in robotic systems. Bramblett and Bezzo [15, 16] show that a particle-based approximation of the Kripke model, combined with DEL product update semantics for belief revision, enables efficient multi-robot exploration and task allocation in communication-restricted environments, matching the performance of idealized fully-connected systems. Bramblett, Reasoner, and Bezzo [17] extend this to implicit coordination without any communication, using active inference over higher-order DEL belief structures to let robots signal intent through motion and resolve assignment ambiguities. Löwe, Pacuit, and Witzel [18] provide the formal planning problem definitions and structural tractability results (self-absorption, commutativity, bounded model growth) that ground these applied approaches in rigorous theory.

The integration path identified in this survey a Cognitive Layer maintaining the Kripke model, interfacing with an EPDDL planner, and dispatching epistemic actions via ROS2 action servers provides a concrete architectural roadmap. The primary open challenges are the design of tractable epistemic state representations, the development of an EPDDL-to-ROS2 toolchain, and the scaling of higher-order reasoning to larger robot teams under real-time constraints.

## References

---

- [1] Macenski, S., Foote, T., Gerkey, B., Lalancette, C., & Woodall, W. (2022). *Robot Operating System 2: Design, architecture, and uses in the wild*. Science Robotics, 7(66).
- [2] Martín, F., Mateos, J., Vargas, F., Moreno, R., Fernandez-Rodríguez, J. C., & Ginesi, A. (2021). *PlanSys2: A Planning System Framework for ROS2*. In Proceedings of the IEEE/RSJ IROS.
- [3] Baltag, A., Moss, L. S., & Solecki, S. (1998). *The Logic of Public Announcements, Common Knowledge and Private Suspicions*. In Proceedings of TARK-98.
- [4] van Ditmarsch, H., van der Hoek, W., & Kooi, B. P. (2007). *Dynamic Epistemic Logic*. Volume 337. Springer Netherlands.
- [5] Burigana, A., & Fabiano, F. (2026). *The Epistemic Planning Domain Definition Language: Official Guideline*. Official Guideline Reference for the Epistemic Planning Track at IPC-26.
- [6] Bolander, T., & Andersen, M. B. (2011). *Epistemic Planning for Single- and Multi-Agent Systems*. Journal of Applied Non-Classical Logics, 21(1), 9–34.
- [7] Coles, A., Coles, A., Fox, M., & Long, D. (2010). *Forward-Chaining Partial-Order Planning*. In Proceedings of ICAPS.
- [8] Faconti, D., & Colledanchise, M. (2018). *BehaviorTree.CPP: A Behavior Tree Library for C++*. Available: <https://www.behaviortree.dev>.
- [9] Open Robotics (2021). *Open-RMF: Open Robotics Middleware Framework*. Available: <https://www.open-rmf.org>.
- [10] Muise, C., Belle, V., Felli, P., McIlraith, S. A., Miller, T., Pearce, A. R., & Sonenberg, L. (2015). *Planning Over Multi-Agent Epistemic States: A Classical Planning Approach*. In Proceedings of AAAI 2015.
- [11] Son, T. C., Pontelli, E., Baral, C., & Gelfond, G. (2014). *Finitary S5-Theories*. In Proceedings of JELIA 2014. LNCS vol. 8761, Springer.
- [12] Kominis, F., & Geffner, H. (2015). *Beliefs in Multiagent Planning: From One Agent to Many*. In Proceedings of ICAPS 2015.
- [13] Hintikka, J. (1962). *Knowledge and Belief: An Introduction to the Logic of the Two Notions*. Cornell University Press.
- [14] Ghallab, M., Nau, D. S., & Traverso, P. (2004). *Automated Planning: Theory and*

*Practice*. Elsevier.

- [15] Bramblett, L., Gao, S., & Bezzo, N. (2023). *Epistemic Planning for Heterogeneous Robotic Systems*. arXiv:2308.00579. In Proceedings of the IEEE International Conference on Robotics and Automation (ICRA).
- [16] Bramblett, L., & Bezzo, N. (2023). *Epistemic Planning for Multi-Robot Systems in Communication-Restricted Environments*. *Frontiers in Robotics and AI*, 10:1149439. doi:10.3389/frobt.2023.1149439.
- [17] Bramblett, L., Reasoner, J., & Bezzo, N. (2025). *Implicit Coordination using Active Epistemic Inference for Multi-Robot Systems*. arXiv:2501.03907.
- [18] Löwe, B., Pacuit, E., & Witzel, A. (2010). *Planning Based on Dynamic Epistemic Logic*. In Proceedings of the Workshop on Logic and the Simulation of Interaction and Reasoning (LSIR), IJCAI 2009.